Chapter 3 Pandas

Data Analytics Using Python

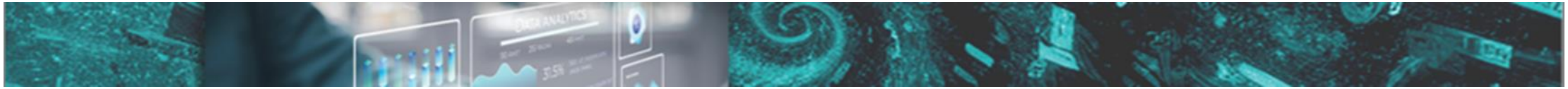
Dr. Ziping Liu

Chapter 3 Pandas

https://he.kendallhunt.com/product/data-analytics-using-python

Pandas

- a Python package specifically designed for data analysis and manipulation
- primary data structures: Series and DataFrames which are structured like spreadsheets.
- provides various operations to make data manipulation and analysis easy and fast, especially for large and complex datasets



Introduction to Pandas

- Why Pandas?
  - NumPy excels in performing intensive scientific computing with large numeric datasets
  - For data analysis applications that involve heterogeneous types of large datasets—traditionally managed by spreadsheets—Pandas offers distinct advantages

Operations spreadsheets provide: data loading, data analysis, data visualization, data modeling

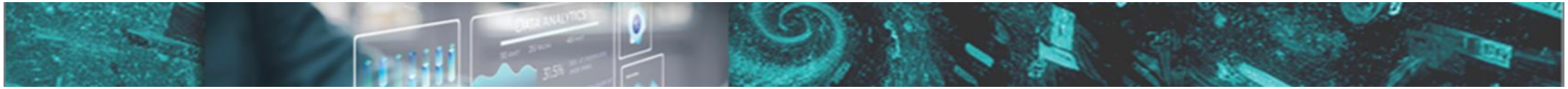
- Pandas can perform all the operations that spreadsheets do, and it also offers capabilities beyond spreadsheets, such as advanced data manipulation and complex modeling with machine learning algorithms.

Efficiency and scale

- Pandas is more efficient than spreadsheets when processing large and complex datasets.
- Pandas can handle data that spreadsheets cannot manage effectively.

Data processing

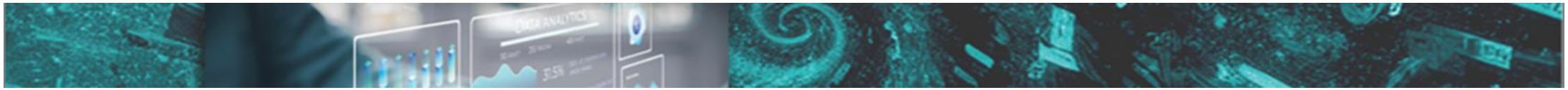
- Pandas provides a wide range of operations, including handling missing data, filtering, sorting, grouping, and merging, which are often cumbersome in spreadsheets.
- Pandas also offers advanced functions for time series analysis, aggregations, and other tasks that spreadsheets cannot manage effectively.



Introduction to Pandas

- Installation and Testing

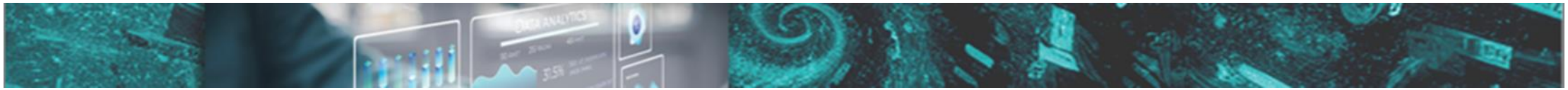
Using pip:	Using conda:
pip install pandas	conda install pandas
Verifying the installation: import pandas as pd print(pd.__version__)	
Testing the Pandas installation using a simple script: import pandas as pd stock_data_dict = {'ticker': ['AAPL', 'TSLA', 'GOOG'], 'price': [150.23, 900.00, 2800.00]} df = pd.DataFrame(stock_data_dict) print(df)	



Getting Started with Pandas

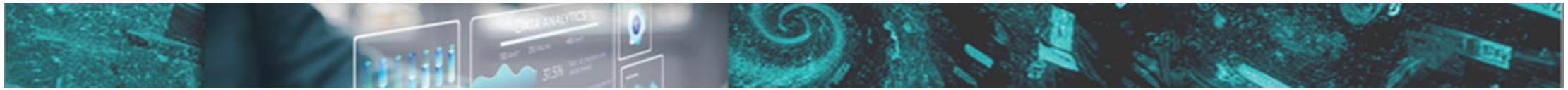
- Pandas DataFrames/Series
  - Series: one-dimensional labeled array, where labels serve as indices
  - DataFrame extends the concept of a Series to two dimensions, with both rows and columns labeled (row index and column index)
- Pandas DataFrames/Series vs NumPy ndarray

	Pandas's Series and DataFrames:	NumPy ndarray:
speed	faster with element-wise ops	Faster with element-wise ops
data types	heterogenous	homogeneous
size	can be altered with functions such as append(), drop(), and insert()	fixed once created
dimension	Series is one-dimension DataFrame is two-dimension	Can be one-dimension, two-dimension or n-dimension



Use Pandas, create Series and Dataframes

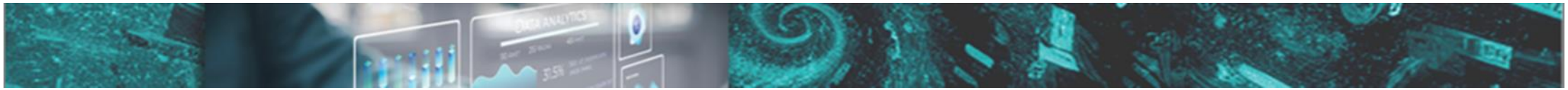
- use Pandas: import pandas as pd
- create a Series: see textbook example
  - use the series method with the default index
  - use the series method with a specified index
  - use the series method with a dictionary
- create a DataFrame: see textbook example
  - use DataFrame method with a dictionary, a list, or a two-dimensional NumPy array
  - can specify names for both the index and columns
  - create a DataFrame from a file, such as a .csv(Comma-Separated Values) file



Viewing and Inspecting on DataFrames

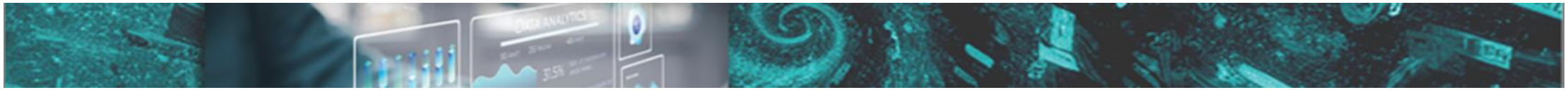
- a summary view of the data, with details on the first and last records, general statistics, and metadata

DataFrame description function	Description
head(n)	display the top n entries of a DataFrame
tail(n)	display the last n entries of a DataFrame
describe()	provide general statistics about a DataFrame
info()	provide the total number of entries, data types for each column and the number of non-null entries for each column



Indexing and slicing

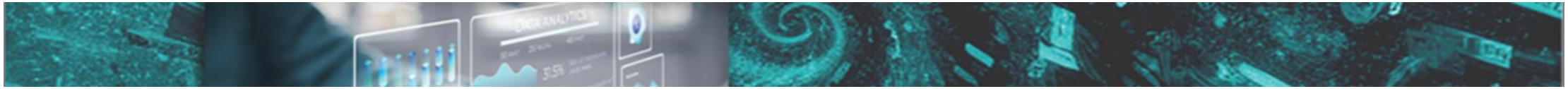
Indexing and slicing methods	descripitons
df[label]	select a single column by its label
.loc[row label, col label]	select a single value based on row and column labels, is preferable for DataFrames with labeled indices and columns
.loc[starting row label:ending rowlabel, starting column label:ending column label]	select range of contiguous row and column data based on row and column labels. Slicing is inclusive
.iloc[row position number, column position number]	select a single value based on integer positions (0-based indexing)
.iloc[starting row position number:ending row position number, starting column position number:ending column position number]	select range of contiguous row and column data based on integer positions. Slicing is inclusive
.at[row label, col label]	access a single value based on row and column labels, similar to .loc but more efficient for scalar access
.iat[row position number, column position number]	access a single value based on integer positions, similar to .iloc but more efficient for scalar access



Data Manipulation

- Data Filtering: select data based on specific conditions

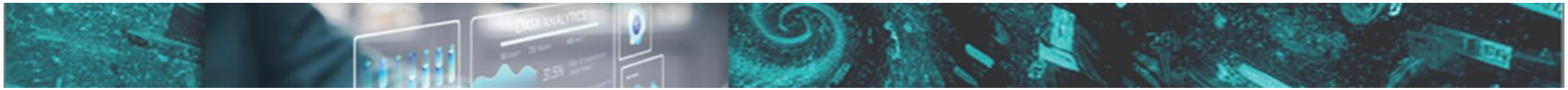
Filtering methods	Descriptions
boolean indexing	write boolean expressions to filter rows
.query method	write SQL-like filtering expressions with column names, comparison operators, and logical operators (AND, OR, NOT)
.isin method	provide specific values to filter rows based on membership in a list or set



Data Manipulation

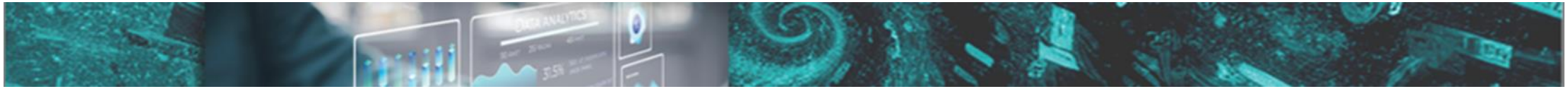
- Handling Missing Data: to locate, remove, or replace

methods handling missing values	descriptions
.info()	provide the total number of entries, data types for each column and the number of non-null entries for each column
.isnull()	returns a DataFrame filled with boolean values. Used to locate the location with null values.
.dropna(axis=0, how='any')	axis can be 0 or 1, 0 is default and is for index, 1 is for columns. how can be “any” or “all”, “any” is default. Used to remove the entries with null values.
.fillna(inplace=False)	inplace can be True or False, False is default. Returns a new DataFrame if False, otherwise, alter the original DataFrame. Used to replace null values with other values.



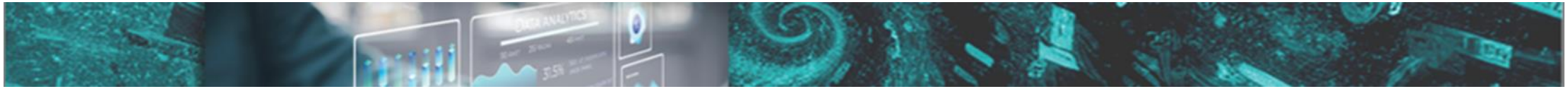
Data Manipulation

- Aligning Data
  - How to apply element-wise operations on DataFrames with different shapes?
  - DataFrames can be aligned based on their indexes before performing operations.
  - methods such as .align() and .reindex() can adjust the data according to the index and column labels



Data Manipulation

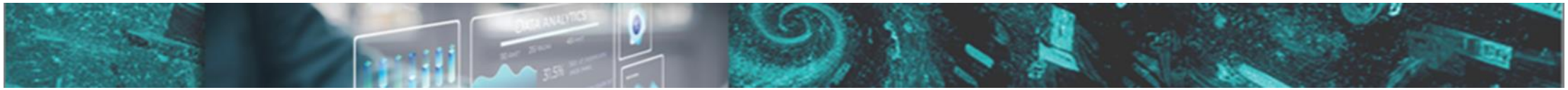
- Operations on DataFrames
  - Arithmetic operations
    - two DataFrames with the same shape: arithmetic operations directly applied on the corresponding elements
    - two DataFrames with different shapes: broadcasting or realignment using .align() and .reindex() first
    - add a Series to a DataFrame: automatically aligns the Series with the DataFrame's index



Data Manipulation

- Operations on DataFrames
  - Apply customized operations

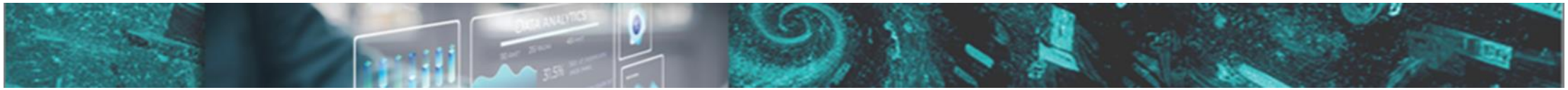
function	description
map()	used primarily for element-wise transformations in a Series, either by applying a function or using a dictionary for value mapping. def function_name(param): function body series_variable.map(function_name or dictionary name) or series_variable.map(lambda v: expression on v)
apply()	used for applying a user-defined function along an axis (0 for rows, 1 for columns) of a DataFrame. It supports complex operations. def function_name(param): function body df.apply(function_name, axis = 0 or 1) or df.apply(lambda v: expression on v, axis = 0 or 1)



Data Manipulation

- Operations on DataFrames
  - Sorting Data

function	description
sort_index(axis=0 or 1)	sort a DataFrame by its index labels
sort_values(by=column_name, ascending=True or False)	sort by the values in one or more columns
apply(sorting_functin, axis = 0 or 1)	sort by a DataFrame's rows



Data Manipulation

- Operations on DataFrames
  - Discretization and Binning: divide continuous data into discrete segments or bins and categorize the data accordingly

function	description
pandas.cut(series, bins=bins, labels=labels, right=True or False)	create bins based on user-defined edges and categorize the data accordingly. returns a categorical data type.
pandas.qcut(series, q=numbers, labels=labels)	create bins based on quantiles, dividing the data into equal-sized segments. return a categorical data type.